Database Management System

Standard Query Language

---

1) **Aggregate Functions:** Used to perform calculations on a group of rows and return a single value.

Common Aggregate Functions

Function	Description
COUNT()	Counts rows
SUM()	Adds values
AVG()	Calculates average
MIN()	Finds minimum value
MAX()	Finds maximum value

Syntax:

```
SELECT AGG_FUNCTION(column_name)
FROM table_name;
```

Example Table: students

id	name	department	marks
1	Rahim	CSE	85
2	Karim	CSE	90
3	Simon	EEE	70
4	Mahim	EEE	80

Queries

```
-- Count total students
SELECT COUNT(*) FROM students;

-- Sum of marks
SELECT SUM(marks) FROM students;

-- Average marks
SELECT AVG(marks) FROM students;

-- Min marks
SELECT MIN(marks) FROM students;

-- Max marks
SELECT MAX(marks) FROM students;
```

Practice:

Question 1: What is the total marks, lowest marks, and number of students in the EEE department?

```
SELECT
    SUM(marks) AS total_marks,
    MIN(marks) AS lowest_marks,
    COUNT(id) AS number_of_students
FROM students
WHERE department = 'EEE';
```

Question 2: Find the highest marks scored in the CSE department and how many students scored above 85.

```sql
SELECT
    MAX(marks) AS highest_marks,
    COUNT(id) AS students_above_85
FROM students
WHERE department = 'CSE' AND marks > 85;
```

Question 3: What is the average marks and total count of students across all departments except CSE?

```sql
SELECT
    AVG(marks) AS average_marks,
    COUNT(id) AS total_students
FROM students
WHERE department != 'CSE';
```

2) **WHERE Clause:** Filters rows before grouping.

Syntax

```
SELECT * FROM table_name
WHERE condition;
```

Examples

```
-- Students with marks > 80
SELECT * FROM students
WHERE marks > 80;

-- Students from CSE department
SELECT * FROM students
WHERE department = 'CSE';

-- Multiple conditions
SELECT * FROM students
WHERE marks > 80 AND department = 'CSE';
```

3) **LIKE Pattern:** The LIKE operator is a logical operator that tests whether a string contains a specified pattern or not. MySQL provides two wildcard characters for constructing patterns:
   1. **% (percentage):** wildcard matches any string of zero or more characters.
   2. **_ (underscore):** wildcard matches any single character at its position
   3. **Structure:**

```
SELECT *
FROM table_name
WHERE column_name LIKE pattern;
```

The Formation of the % Operator in Pattern Matching

1. **Prefix Match (%x):** Matches any sequence of characters before x.
   Example: %sh Matches ash, cash, bash, mash, lash, splash, etc.
2. **Suffix Match (x%):** Matches any sequence of characters after x.
   Example: a% Matches a, apple, ant, abracadabra, antelope, etc.
3. **Substring Match (%x%):**Matches any sequence of characters before and after x.
   Example: %on% Matches Jefferson, Monir, Onie, Onion, etc.

The Formation of the _ (Underscore) Operator in Pattern Matching

The _ operator in SQL pattern matching is used to represent a single character at a specific position. Unlike the % wildcard, the _ operator requires an exact match for both the position and the total length of the pattern. Here's how it works:

1. **Prefix Match (_xxxx...)**
   - Meaning: The underscore at the beginning represents exactly one character, followed by a fixed series of characters that define the rest of the pattern.
   - Example:
     - Pattern _am → Matches: Cam, Ram, bam, ham (any 3-letter word ending in "am").
     - Not Accepted: Scam (4 letters, exceeds length 3) or am (2 letters, too short).
     - Pattern _ _sh → Matches: cash, bash, mash, lash (any 4-letter word ending in "sh").
     - Not Accepted: Ash (3 letters) or splash (6 letters).
2. **Suffix Match (xxx...xx_)**
   - Meaning: The underscore at the end represents one character, while the preceding part of the pattern is fixed.
   - Example:
     - Pattern Ri_ _ → Matches: Rise, Ripe, Ride (any 4-letter word starting with "Ri").
     - Not Accepted: Risen (5 letters) or Ri (2 letters).
3. **Mixed Positions (X_x__X___X)**
   - Meaning: Underscores can appear anywhere in the pattern. Each _ still represents exactly one character, and the overall length must match exactly.
   - Example:
     - Pattern L_m_ → Matches: Lime, Lame, Lomo (any 4-letter word starting with "L", followed by any char, then "m", then any char).
     - Not Accepted: Lemon (5 letters) or Lam (3 letters).

Examples:

```
-- Names starting with 'R'
SELECT * FROM students
WHERE name LIKE 'R%';

-- Names ending with 'm'
SELECT * FROM students
WHERE name LIKE '%m';

-- Names containing 'ar'
SELECT * FROM students
WHERE name LIKE '%ar%';

-- Names with exactly 4 characters
SELECT * FROM students
WHERE name LIKE '____';
```

Specific example:

a) Match any email that ends with the same domain

```
SELECT *
FROM students
WHERE email LIKE '%@bcse.uiu.ac.bd';
```

b) Match emails where the **first character is 'm' and next two characters can be anything**

```
SELECT *
FROM students
WHERE email LIKE 'm__%';
```

4) **GROUP BY:** GROUP BY is an SQL clause used to arrange identical data into groups, based on one or more columns. It is typically used with aggregate functions (COUNT, SUM, AVG, MAX, MIN) to perform calculations on each group separately.

Structure:

```
SELECT column_name, AGG_FUNCTION(column_name)
FROM table_name
GROUP BY column_name;
```

Example

```
-- Count students per department
SELECT department, COUNT(*)
FROM students
GROUP BY department;

-- Average marks per department
SELECT department, AVG(marks)
FROM students
GROUP BY department;
```

5) **ORDER BY:** The ORDER BY clause is used in SQL to sort the result set of a query by one or more columns. You can specify the sort order as: ASC (ascending): default, from lowest to highest, DESC (descending): from highest to lowest

Question 1: List all departments and their average salaries, sorted by average salary from highest to lowest.

```
SELECT dept_name, AVG(salary) AS avg_salary
FROM faculty
GROUP BY dept_name
ORDER BY avg_salary DESC;
```

6) **HAVING Clause:** The HAVING clause is an SQL clause used to filter groups after they have been filtered by the GROUP BY clause. It applies conditions to aggregated data, unlike WHERE which filters individual rows. WHERE filters rows, HAVING filters groups.

Examples

```
-- Departments with more than 1 student
SELECT department, COUNT(*)
FROM students
GROUP BY department
HAVING COUNT(*) > 1;

-- Departments with average marks > 80
SELECT department, AVG(marks)
FROM students
GROUP BY department
HAVING AVG(marks) > 80;
```

7) WHERE vs HAVING

Feature	WHERE	HAVING
Filters	Rows	Groups
Used with aggregates	No	Yes
Runs before GROUP BY	Yes	No (runs after GROUP BY)

8) Set Operations: Used to combine results of two queries.

Operator	Description
UNION	Removes duplicates
UNION ALL	Keeps duplicates
INTERSECT	Common rows
EXCEPT / MINUS	Rows of first query not in second

Example Tables
Table 1: cse_students

id	name	year
1	Abir	3
2	Bahar	2
3	Samin	4
4	Depu	1
5	Evan	3

Table 2: eee_students

id	name	year
1	Tanzim	2
2	Evan	1
3	Nahian	3
4	Abir	4
5	Aayan	3

Queries

```
-- Union (no duplicates), Return all the unique students from both table avoiding duplicates
SELECT name FROM cse_students
UNION
SELECT name FROM eee_students;

-- Union All, Return all the students from both table with duplicates
SELECT name FROM cse_students
UNION ALL
SELECT name FROM eee_students;

-- Intersection, Returns names that appear in both tables.
SELECT name FROM cse_students
INTERSECT
SELECT name FROM eee_students;

-- Difference Returns names that are in cse_students but NOT in eee_students.
SELECT name FROM cse_students
EXCEPT
SELECT name FROM eee_students;
```